



SQLite-Carver-Pro

Official Forensic Guide & Technical Reference Manual

Version 1.3.0 — Forensic Analysis Edition

Core Technical Capabilities Covered in this Guide:

- Low-Level SQLite B-Tree Parsing & O(1) Allocation Math
- Mathematical Foundation of Forensic Confidence Scoring (0.00 to 1.00)
- Freeblock traversal, Slack Space, Unallocated margins & Freelist
- Resilient Truncated String Recovery inside Cell Slack
- Multi-Format Forensic Timestamp Engine (Unix s/ms/μs, WebKit, FILETIME, Mac Cocoa, GPS)
- Heuristic BLOB File Signature Analysis & Carving (--dump-blobs)
- WAL-Only Table Isolation & Mutation Detection (★ WAL-ONLY TABLE)
- Dynamic Protocol Buffers wire decoding & False-Positive Elimination
- Interactive WAL Timeline with Visual Before/After Diffs (--table filter)
- Dual Language Support (English & French) & Unified Export Router

Author: Dam-FOR3K (with technical assistance from Antigravity / Google DeepMind)

Compatibility: Windows 64-bit / Linux / macOS (Python 3.11+ & Standalone .exe)

Publication Date: September 2026 (Edition v1.3.0)

Table of Contents

1. Mandatory Verification Advisory & Legal Disclaimer

2. Introduction & Architecture Overview

3. SQLite Storage Internals & Forensic Carving Mechanisms

- 3.1. Physical Layout of a SQLite Database (.db)
- 3.2. B-Tree Page Types & Varint Encoding
- 3.3. The 4 Forensic Carving Zones (Exact Slack Interval Calculations)
- 3.4. Multi-Page Overflow Chain Reassembly
- 3.5. Write-Ahead Log (WAL) Journaling & Automatic Merging in Carve
- 3.6. Automated Embedded Binary Payload Decoders (Apple bplist, Protobuf, zlib)
- 3.7. Multi-Format Forensic Timestamp Engine (Unix s/ms/μs, WebKit, FILETIME, Cocoa, GPS)
- 3.8. BLOB File Signature Analysis & Heuristic Carving (Images, PDF, ZIP, SQLite)
- 3.9. WAL-Only Table Isolation and Detection

4. Understanding the Forensic Confidence Score (Confidence Threshold)

- 4.1. Why is a Confidence Score Essential?
- 4.2. Mathematical Formula, SQLite Invariants & Nested Structures Scale
- 4.3. Interpretation Guide for the 4 Confidence Tiers
- 4.4. Selecting the Optimal Threshold by Investigation Scenario

5. Command Line Interface & Standalone Executable Guide

- 5.1. Database Inspection (`info`)
- 5.2. Deleted Record Carving & BLOB Dumping (`carve --dump-blobs`)
- 5.3. Deep Forensic Search (`search`)
- 5.4. Transaction Diffing & Timeline (`wal-diff` with Table Filter)
- 5.5. Independent Binary Microscope (`decode-blob`)

6. Forensic Reports & Export Formats

- 6.1. The Dual-View Interactive HTML Dashboard (All Evidence & WAL Timeline with 60 FPS Pagination, Decoded Timestamp Badges & Language Switcher)
- 6.2. Indented JSON (`.json`) & JSON Lines (`.jsonl`)
- 6.3. Excel Tabular CSV (`.csv`) & Apache Parquet (`.parquet`)

7. Python API for Automated Pipelines

1. Mandatory Verification Advisory & Legal Disclaimer

⚠ Essential Methodological Standard

In any digital forensic analysis, **no critical, operational, or legal decision should ever be based solely on automated heuristic reconstructions**. Data carved from freeblocks, cell slack space, or uncommitted Write-Ahead Logs may have been partially overwritten or modified by subsequent operations. Forensic practitioners must independently corroborate all extracted records, decoded timestamps, and file artifacts against primary evidence sources and operating system logs before forming conclusions. The software is provided "as is", without warranty of any kind.

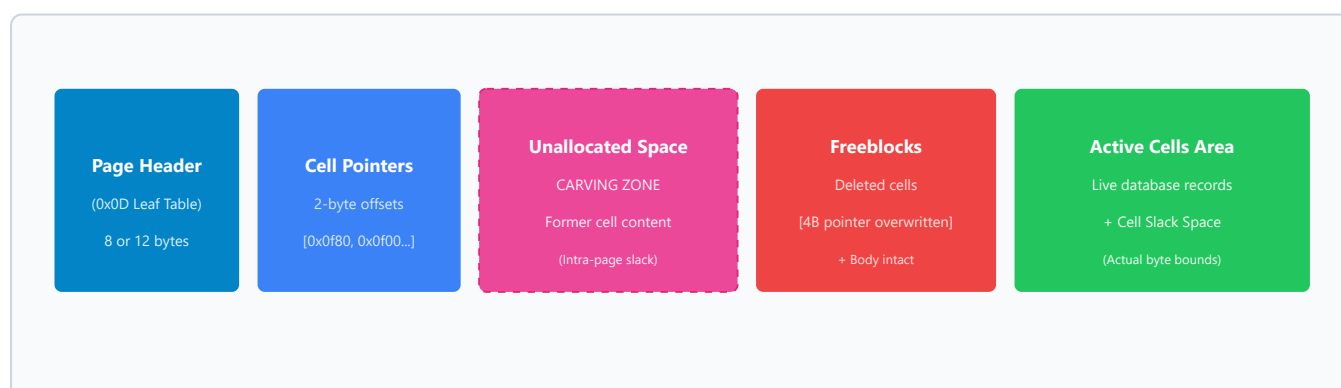
2. Introduction & Architecture Overview

SQLite databases constitute one of the most critical evidentiary containers in digital investigations. They store web browser histories, encrypted messaging artifacts (Signal, WhatsApp, Telegram), mobile operating system caches (iOS and Android), system diagnostics, and authentication tokens.

Conventional carving tools typically employ brute-force regular expressions or plain-text string dumping, losing the relational context, table mappings, data types, and binary payloads embedded in BLOB fields.

SQLite-Carver-Pro v1.3.0 is engineered as a comprehensive system-level forensic engine that parses binary B-Tree pages, extracts deleted cell remnants from unallocated space and freeblocks, reassembles multi-page overflow chains, decodes nested payloads (Apple bplist, Protobuf, zlib), translates multi-format timestamps into UTC, and isolates files embedded in BLOB columns.

Visual Anatomy of a SQLite Leaf Table Page (4096B)



3. SQLite Storage Internals & Forensic Carving Mechanisms

3.1. Physical Layout of a SQLite Database (.db)

A SQLite database file is divided into uniform fixed-size pages (typically 4096 bytes, or 512 to 65536 bytes). Page 1 commences with the 100-byte database header.

3.2. B-Tree Page Types & Varint Encoding

SQLite records serialize variable-length data using **Varints** (1 to 9 bytes in Big-Endian) and a **Serial Type Table** to decode columns:

Serial Type	Data Type & Byte Size	Forensic Significance
0	0 bytes	NULL value.
1, 2, 3, 4, 5, 6	1, 2, 3, 4, 6, 8 bytes	Signed integers (with sign extension for 24-bit and 48-bit ints).
7	8 bytes	IEEE 754 64-bit floating-point value.
8, 9	0 bytes	Integer constants 0 or 1 (engine optimization).
N ≥ 12 (Even)	(N - 12) / 2 bytes	Raw binary BLOB (e.g. images, bplist, protobuf).
N ≥ 13 (Odd)	(N - 13) / 2 bytes	UTF-8 encoded TEXT string.

3.6. Automated Embedded Binary Payload Decoders

Modern mobile and desktop applications routinely store complex serialized structures inside BLOB columns:

- **Apple Binary Plist (bplist00)**: Hierarchical extraction of dictionaries, arrays, strings, dates, and integers.
- **Google Protocol Buffers**: Dynamic wire parser extracting field numbers and values without preliminary .proto files, with strict false-positive rejection.
- **Zlib / Deflate Streams**: Automated decompression and identification of inner plain text or secondary structures.

3.7. Multi-Format Forensic Timestamp Engine (v1.3.0)

Application databases store timestamps using disparate numeric representations, complicating chronological event reconstruction. **SQLite-Carver-Pro v1.3.0** incorporates an automated multi-format timestamp decoding engine (`sqlite_carver.decoders.timestamps`) capable of detecting and translating integers into ISO 8601 UTC timestamps across the following formats:

Timestamp Format	Epoch & Units	Typical Range (2020-2030)	Forensic Origin
Unix Epoch (Seconds)	Jan 1, 1970 (seconds)	1.6e9 - 1.9e9 (10 digits)	Linux systems, REST APIs, native SQLite
Unix Epoch (Milliseconds)	Jan 1, 1970 (milliseconds)	1.6e12 - 1.9e12 (13 digits)	JavaScript, Android SQLite, Telegram
Unix Epoch (Microseconds)	Jan 1, 1970 (microseconds)	1.6e15 - 1.9e15 (16 digits)	High-resolution databases, kernel logs
WebKit / Safari / Chrome	Jan 1, 1601 (microseconds)	1.3e16 - 1.4e16 (17 digits)	Google Chrome History, Chromium, Safari
Windows FILETIME	Jan 1, 1601 (100-nanoseconds)	1.3e17 - 1.4e17 (18 digits)	NTFS attributes, Windows Registry, Skype, Teams
Apple Cocoa Absolute Time	Jan 1, 2001 (seconds)	6.0e8 - 9.0e8 (9 digits)	iOS Photos, Apple Messages, macOS Safari
GPS Time	Jan 6, 1980 (seconds)	1.2e9 - 1.5e9 (10 digits)	Satellite navigation telemetry, geolocation logs

The engine employs column-name heuristics (`time` , `date` , `created` , `timestamp` , `sent`) and strict calendar bounds (1995–2040) to prevent false positives.

3.8. BLOB File Signature Analysis & Heuristic Carving (v1.3.0)

Binary columns frequently embed complete standalone files (profile pictures, attachments, cached databases). The module `sqlite_carver.decoders.blobs` inspects leading bytes against known magic signatures:

- **Images:** PNG (`\x89PNG\r\n\x1a\n`), JPEG (`\xff\xd8\xff`), GIF (`GIF87a` , `GIF89a`), WebP (`RIFF...WEBP`), TIFF (`II*\x00` ou `MM*\x00*`).
- **Documents & Archives:** PDF (`%PDF-`), ZIP / APK / DOCX (`PK\x03\x04`), GZIP (`\x1f\x8b`).
- **Embedded Databases:** SQLite Database Header (`SQLite format 3\x00`).

The CLI option `--dump-blobs <directory>` exports all detected embedded files directly to disk for independent verification.

3.9. WAL-Only Table Isolation and Detection (v1.3.0)

In some forensic scenarios, tables may be created and modified exclusively in the Write-Ahead Log journal without having been written back to the base database. **SQLite-Carver-Pro v1.3.0** flags these occurrences with a prominent `[★ WAL-ONLY TABLE]` status in both the timeline diff and report exports.

4. Understanding the Forensic Confidence Score (Confidence Threshold)

4.1. Why is a Confidence Score Essential?

When examining damaged databases or scanning unallocated page margins and freeblocks, disks contain thousands of residual bytes. Without disciplined validation, random byte sequences could be mistakenly parsed as legitimate SQLite records (false positives).

To ensure **analytical integrity**, `sqlite-carver-pro` assigns each carved record a **Confidence Score between 0.00 and 1.00**.

4.2. Mathematical Formula, SQLite Invariants & Nested Structures Scale

The confidence score is computed dynamically across 5 strict criteria:

1. **Physical Payload Invariant:**

The physical formula `payload_size = header_size + sum(column_lengths)` must hold exactly. When a valid record is identified, the carver advances its scanner by the total cell size to prevent phantom duplicate detections.

2. **Printable Character Validation (Sanity Check):**

For every column typed as `TEXT` (odd serial type ≥ 13), the engine verifies that 100% of bytes constitute valid printable UTF-8 characters. If binary noise is present, the score drops drastically.

3. **Heuristic Schema Matching & Intrinsic Score:**

The engine matches discovered serial types against schemas from `sqlite_schema`. If no schema is available (e.g. dropped tables), an intrinsic heuristic score is assigned based on structural coherence.

4. **Overwritten Header Penalty (Freeblock Offset):**

SQLite overwrites the first 4 bytes of freeblocks with its freelist offset pointer. The engine reconstructs cells starting at offset + 4 and applies an initial penalty (score set to `0.85`).

5. **Calibrated Scale for Embedded BLOBs:**

- **Apple Binary Plist (`bplist00`):** `1.00` (Strict magic signature + certified parser).
- **Zlib Stream + Plain Text:** `0.95` (CRC32/Adler32 verified + printable UTF-8).
- **Zlib Stream + Protobuf / Binary:** `0.85` (CRC verified).
- **Google Protocol Buffers (Multi-field):** `0.85` (Validated wire types & field tags).
- **Google Protocol Buffers (Single field):** `0.65` (Conservative score to eliminate false positives).

4.3. Interpretation Guide for the 4 Confidence Tiers

< 0.50 (Noise / Rejected)	0.50 - 0.64 (Slack)	0.65 - 0.84 (Freeblock)	0.85 - 1.00 (High Fidelity)
---------------------------	---------------------	-------------------------	-----------------------------

Score Range	Confidence Tier	Typical Evidentiary Origin & Significance
0.85 – 1.00	High Fidelity (Maximum Certainty)	Live active cells or recently deleted intact records. Perfect schema correspondence, strict column types, 100% printable text.
0.65 – 0.84	Good Confidence (Reconstructed Rows)	Cells carved from Freeblocks with overwritten headers, partial rows with NULL fields, or verified Protobuf structures.
0.50 – 0.64	Moderate Confidence (Slack & Relics)	Fragments recovered from Cell Slack Space or unreferenced Freelist Pages (tables dropped via <code>DROP TABLE</code> whose definitions no longer exist in <code>sqlite_schema</code>).
< 0.50	Binary Noise (Rejected by default)	Fragmented artifacts, random encryption key bytes, or compression leftovers. Filtered out automatically.

5. Command Line Interface & Standalone Executable Guide

5.1. Database Inspection ([info](#))

```
# Low-level diagnostic, freelist detection, and schema table listing
sqlite-carver-v1.3.0.exe info evidence.db
```

```
Command Prompt - sqlite-carver-v1.3.0.exe info (v1.3.0)

SQLite-Carver-Pro v1.3.0
Forensic Parser, Slack Carver, Freelist & WAL Diff Engine

[bold green]SQLite Database Header Analysis[/bold green]: Page Size 4096B, Version 2 (WAL), Freeli
Discovered 3 Freelist Page(s): [6, 7, 5]
[Discovered Schema Tables]: users (Page 2), exfiltration_logs (Page 4), shadow_bank (Page 5)
```

5.2. Forensic Deleted Record Carving & BLOB Dumping ([carve](#))

v1.3.0 Feature: The `--dump-blobs <directory>` argument automatically carves and dumps embedded files (JPEG, PNG, PDF, ZIP, SQLite, etc.) into the specified output folder. Numeric timestamp values are automatically annotated with decoded UTC strings.

```
# 1. Carve all records with automatic embedded BLOB extraction
sqlite-carver-v1.3.0.exe carve evidence.db --dump-blobs ./extracted_blobs

# 2. Isolate ONLY carved deleted records (Freeblocks, Freelist, Slack, WAL)
sqlite-carver-v1.3.0.exe carve evidence.db --deleted-only

# 3. Filter by strict confidence threshold (e.g. >= 0.80 to exclude noisy slack)
sqlite-carver-v1.3.0.exe carve evidence.db --min-confidence 0.80

# 4. Generate the full interactive web report with decoded timestamp badges
sqlite-carver-v1.3.0.exe carve evidence.db --export forensic_report.html
```

5.3. Deep Forensic Search ([search](#))

```
# Recursive evidence search across database + WAL journal + dump found BLOBs
sqlite-carver-v1.3.0.exe search evidence.db "session_token_xyz" --dump-blobs ./carved_evidence

# Explicitly disable companion WAL search
sqlite-carver-v1.3.0.exe search evidence.db "token" --no-include-wal

# Search hex pattern inside binary BLOB fields
sqlite-carver-v1.3.0.exe search evidence.db "deadbeef" --hex
```

5.4. Transaction Diffing & Timeline (wal-diff)

```
# Chronological WAL transaction reconstruction with WAL-Only Table indicators
sqlite-carver-v1.3.0.exe wal-diff evidence.db evidence.db-wal

# Target a specific table and export timeline
sqlite-carver-v1.3.0.exe wal-diff evidence.db evidence.db-wal --table users --export wal_diff.html
```

6. Forensic Reports & Export Formats

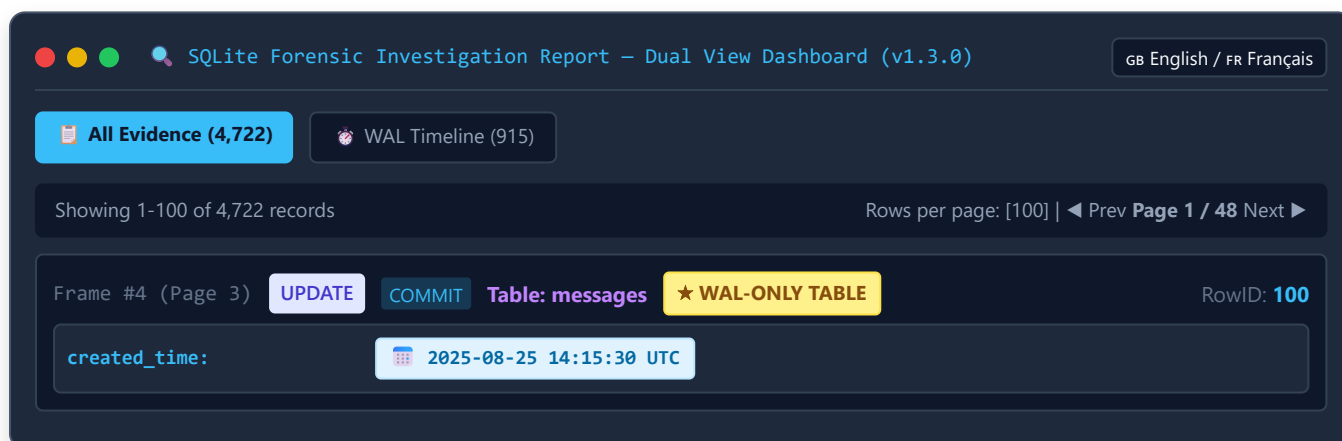
6.1. The Dual-View Interactive HTML Dashboard (All Evidence & WAL Timeline with 60 FPS Pagination, Decoded Timestamp Badges & Language Switcher)

The HTML export generates a **100% Standalone Air-Gapped Web Application** that opens directly in any modern browser (Edge, Chrome, Firefox, Safari) without dependencies.

Visual Enhancements in v1.3.0:

- **Timestamp Badges:** Every numeric value recognized as a valid timestamp is rendered with a clean calendar badge [ 2026-08-25 14:15:30 UTC] alongside its raw integer.
- **File BLOB Previews:** Detected file payloads (PNG, JPEG, PDF, etc.) display their recognized format and file extension.
- **WAL-Only Table Indicator:** Ephemeral tables created strictly in the WAL are highlighted with the gold badge [★ WAL-ONLY TABLE] .

```
sqlite-carver-v1.3.0.exe carve evidence.db --export forensic_report.html
```



6.2. Export Format Reference Guide

Format	Extension	Target Audience & Use Case
HTML Report	<code>.html</code>	Recommended for Analysts. Dual-view interactive report with instant search, confidence slider, 60 FPS pagination, WAL timeline, decoded timestamp badges, and expandable BLOB inspection.
Indented JSON	<code>.json</code>	Human-readable formatted JSON with <code>_timestamps</code> enrichment for VS Code or script ingestion.
JSON Lines	<code>.jsonl</code>	Streaming single-line format optimized for SIEM pipelines (Splunk, ElasticSearch, Cortex XSOAR).
Excel CSV	<code>.csv</code>	Tabular spreadsheet compatible with Microsoft Excel, LibreOffice Calc, or external SQL loaders.
Apache Parquet	<code>.parquet</code>	Columnar binary compressed format for big data forensics (Polars, DuckDB, Pandas).

7. Python API for Automated Pipelines

```
from pathlib import Path
from sqlite_carver import (
    SQLiteCarver, WalDiffEngine, ForensicSearchEngine, export_html,
    TimestampDecoder, detect_file_signature, dump_blob_to_file
)

# 1. Programmatic carving from raw bytes (v1.3.0)
raw_db = Path("evidence.db").read_bytes()
carver = SQLiteCarver(raw_db)

# 2. Extract records filtered by confidence (e.g. >= 0.80)
records = [r for r in carver.carve_all() if r.confidence >= 0.80]
for r in records:
    if r.source != "active":
        print(f"[{r.source.upper()}] Page {r.page_id} | Table: {r.matched_table} | Conf: {r.confidence}")

# 3. Decode multi-format forensic timestamps
ts_decoder = TimestampDecoder()
decoded = ts_decoder.decode(13398765432000000, column_name="visit_time")
if decoded:
    print(f"Decoded WebKit timestamp: {decoded.iso_string} UTC ({decoded.format_name})")

# 4. Carve and dump embedded BLOB files
for r in records:
    for idx, val in enumerate(r.values):
        if isinstance(val, (bytes, bytearray)):
            sig = detect_file_signature(val)
            if sig:
                print(f"Detected file: {sig.mime_type} ({sig.extension}")
                dump_blob_to_file(val, f"./dump/page_{r.page_id}_col_{idx}")

# 5. Generate interactive bilingual HTML report
export_html(records, "automated_report.html")
```

Recommended DFIR Best Practices

1. Always preserve original evidence integrity by analyzing write-blocked disk images or bit-stream copies.
2. Keep companion `.db-wal` and `.db-shm` files in the same directory for comprehensive transaction timeline diffs.
3. Utilize the self-contained `.html` report format for legal deliverables: it empowers analysts and non-technical stakeholders to interactively inspect evidence.